

M14 SOLO TEACHING BOOK

Kafka + Streaming Foundations | self-contained core teacher

Baseline

Kafka 4.3.1 KRaft local broker; confluent-kafka 2.15.0; Spark Structured Streaming references use the M12 Spark 4.2.0 baseline. External visuals teach mental models; this book controls the course-specific practice and evidence.

DE14-01 - Batch vs streaming

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Confluent - Apache Kafka Architecture Deep Dive](#)

Topics, brokers, partitions, offsets and event-log architecture.

CURRENT / EXACT REFERENCES

[Confluent - Apache Kafka Architecture Deep Dive](#)

[Apache Kafka supported downloads](#)

[Apache Kafka supported downloads](#)

Teacher explanation

Batch processes bounded data at chosen intervals; streaming processes an ongoing event flow with progress and failure state. Streaming is not automatically faster or better: it adds operational complexity. Choose it when latency and continuous reaction justify the cost.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-01 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Classify three requirements as batch or streaming and justify latency/complexity trade-off.

Independent task

Design batch vs streaming choice for payment settlement and defend the latency/SLA boundary.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain batch vs streaming in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed clinic alerts: choose batch/streaming and define latency target.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.

DE14-02 - Events, brokers and topics

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Confluent - Apache Kafka Architecture Deep Dive](#)

Topics, brokers, partitions, offsets and event-log architecture.

CURRENT / EXACT REFERENCES

[Confluent - Apache Kafka Architecture Deep Dive](#)

[Apache Kafka supported downloads](#)

[Apache Kafka supported downloads](#)

Teacher explanation

An event is an immutable fact. Producers append events to topics managed by brokers. A topic is split into partitions; each partition is an ordered append-only log with monotonically increasing offsets. Consumers read; normal consumption does not delete records.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-02 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Run the normal simulator; inspect topic partitions, events and offsets.

Independent task

Design topic/event envelope for parcel status events: key, value, timestamp and retention purpose.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain events, brokers and topics in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed IoT domain: define topic and immutable event fields.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.

DE14-03 - Producers and message keys

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Confluent - Python Producer Class](#)

8-minute visual lesson on producing Kafka events with Python.

CURRENT / EXACT REFERENCES

[Confluent - Python Producer Class](#)

[Confluent Kafka 101 - Partitions](#)

[Apache Kafka supported downloads](#)

Teacher explanation

A producer sends key/value events. The key is not just metadata: partitioning commonly uses it so related events land together. Choose keys from ordering/affinity requirements, not randomly. Producer retries can create duplicates unless idempotence/transactions or downstream deduplication handle them.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-03 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Trace each event key to its partition and prove one key never jumps partitions.

Independent task

Choose a message key for account transactions and explain ordering consequence.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain producers and message keys in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed shipment events: choose key so each shipment stays ordered.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.

DE14-04 - Consumers and safe processing

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Confluent - Python Consumer Class](#)

7-minute visual lesson on reading and processing Kafka events with Python.

CURRENT / EXACT REFERENCES

[Confluent - Python Consumer Class](#)

[Confluent - Consumer Group Protocol](#)

[Apache Kafka supported downloads](#)

Teacher explanation

A consumer polls records, performs work, and advances/commits progress. The dangerous moment is between side effect and offset commit: a crash can cause replay. Safe consumers therefore assume reprocessing can happen and make sinks idempotent or coordinate output/progress atomically.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-04 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Trace the crash-before-offset-commit scenario and show which event replays.

Independent task

Design a consumer that writes to an external database safely when a crash can occur after write but before offset commit.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain consumers and safe processing in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed notification sink: handle crash/replay without duplicate sends.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.

DE14-05 - Partitions and ordering

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Confluent Kafka 101 - Partitions](#)

Focused short visual/lesson: partitions, scalability and within-partition ordering.

CURRENT / EXACT REFERENCES

[Confluent Kafka 101 - Partitions](#)

[Confluent - Kafka durability and ordering guarantees](#)

[Apache Kafka supported downloads](#)

Teacher explanation

Partitions are Kafka's unit of ordering and parallelism. Kafka guarantees order within a partition, not across an entire multi-partition topic. If all events for one customer require order, a stable customer key can keep them together; more partitions increase parallelism but change capacity/operations.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-05 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Verify offsets are monotonic inside each partition and explain why global order is not guaranteed.

Independent task

Given 6 partitions and 9 consumers, predict active/idle consumers and ordering scope.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain partitions and ordering in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed topic has 4 partitions and 6 consumers: predict assignment capacity.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.

DE14-06 - Offsets, group position and replay

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Confluent - Consumer Group Protocol](#)

Consumer groups, partition assignment, committed offsets and restart position.

CURRENT / EXACT REFERENCES

[Confluent - Consumer Group Protocol](#)

[Apache Kafka supported downloads](#)

Teacher explanation

Offsets identify positions inside one partition. A consumer group commits positions so a restarted member can resume. Resetting to earliest/latest changes what unread history is considered. Replay is a feature for recovery/reprocessing, but it can duplicate external side effects unless the consumer is safe.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-06 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Use committed_positions and replay list to reconstruct restart behavior.

Independent task

Write a replay plan from a saved offset without duplicating already-published business effects.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain offsets, group position and replay in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed group starts from a different committed offset: reconstruct replay.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.

DE14-07 - Consumer groups and scaling

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Confluent - Consumer Group Protocol](#)

Consumer groups, partition assignment, committed offsets and restart position.

CURRENT / EXACT REFERENCES

[Confluent - Consumer Group Protocol](#)

[Apache Kafka supported downloads](#)

Teacher explanation

Consumers sharing a group divide partitions so one partition is handled by one group member at a time. Adding consumers beyond partition count creates idle members. Membership changes can rebalance assignments, so processing should tolerate handoff/restart and not keep critical state only in one process memory.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-07 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Inspect assignment of 3 partitions to 4 consumers and explain the idle member.

Independent task

Explain what happens during a group rebalance and what local state must not be assumed durable.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain consumer groups and scaling in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed membership: one consumer dies; explain reassignment and safe state.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.

DE14-08 - Delivery semantics and idempotency

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Confluent - Kafka durability and ordering guarantees](#)

acks, idempotent producer, keyed ordering and failure guarantees.

CURRENT / EXACT REFERENCES

[Confluent - Kafka durability and ordering guarantees](#)

[Confluent - Transactions and guarantees](#)

[Apache Kafka supported downloads](#)

Teacher explanation

At-least-once processing can repeat work after failure; at-most-once can lose work; exactly-once claims are scoped. Kafka idempotent producers and transactions can make Kafka-to-Kafka processing strong, but transactions do not magically cover arbitrary external APIs/databases. State the exact boundary of the guarantee.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-08 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Compare raw processed+replayed operations with unique idempotent sink event IDs.

Independent task

State precisely what “exactly once” does and does not guarantee for Kafka→REST API.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain delivery semantics and idempotency in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed external payment API: define dedup/idempotency boundary.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.

DE14-09 - Serialization, schemas and evolution

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[Confluent - Data Contracts for Long-Term Schema Management](#)

Use 08:38-31:17 for Avro/Protobuf, schema references and compatibility.

CURRENT / EXACT REFERENCES

[Confluent - Data Contracts for Long-Term Schema Management](#)

[Apache Kafka supported downloads](#)

Teacher explanation

Serialization turns event objects into bytes. A schema makes field names/types/contracts explicit. Evolution needs compatibility rules so old/new producers and consumers can coexist. Adding an optional/defaulted field is often safer than removing or changing the meaning/type of a required field.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-09 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Compare schema v1/v2 and classify additive vs breaking change.

Independent task

Classify four schema changes under backward/forward compatibility reasoning.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain serialization, schemas and evolution in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed schema adds nullable device_model then removes event_id: classify safety.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.

DE14-10 - Python producer and consumer workflow

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[DataTalksClub - real-time streaming workshop](#)

Preserved 2026 workshop; use producer/consumer/integration/failure sections only.

CURRENT / EXACT REFERENCES

[Confluent - Python Producer Class](#)

[Confluent - Python Consumer Class](#)

[confluent-kafka 2.15.0 - PyPI](#)

[Apache Kafka supported downloads](#)

Teacher explanation

A Python Kafka workflow has configuration, producer callbacks/flush, consumer polling/error handling, explicit group ID, offset policy and controlled commits. Credentials belong in environment/config, not source. Save topic, partition, offset and event ID evidence so failures can be diagnosed.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-10 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Read producer.py/consumer.py line by line and map every config to behavior.

Independent task

Write your own Python producer/consumer skeleton without copying the reference files.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain python producer and consumer workflow in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed Python domain: produce/consume clinic events with explicit group and manual commit.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.

DE14-11 - Spark Structured Streaming + Kafka

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[DataTalksClub - real-time streaming workshop](#)

Preserved 2026 workshop; use producer/consumer/integration/failure sections only.

CURRENT / EXACT REFERENCES

[Spark 4.2 - Structured Streaming + Kafka Integration](#)

[Spark 4.2 - Structured Streaming Programming Guide](#)

[Apache Kafka supported downloads](#)

Teacher explanation

Spark Structured Streaming can use Kafka as a source/sink while applying DataFrame transformations incrementally. Kafka offsets identify source progress; Spark checkpoint state belongs to the streaming query. Do not delete a checkpoint to “fix” a job without understanding that you are changing its recovery identity.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-11 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Read Spark reference code and identify Kafka source options vs checkpoint location.

Independent task

Design Kafka→Spark transformation with stable checkpoint identity and rerun/restart evidence.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain spark structured streaming + kafka in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed Spark stream: explain consequences of changing/deleting checkpoint.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.

DE14-12 - Lag, failures and streaming operations

PART 1 - UNDERSTAND + WATCH ROUTE

VIDEO FIRST

[DataTalksClub - real-time streaming workshop](#)

Preserved 2026 workshop; use producer/consumer/integration/failure sections only.

CURRENT / EXACT REFERENCES

[Confluent - Consumer Group Protocol](#)

[Spark 4.2 - Structured Streaming Programming Guide](#)

[Apache Kafka supported downloads](#)

Teacher explanation

Consumer lag is the distance between the latest available offset and a group's committed/processed position. Rising lag can mean slow processing, insufficient parallelism, downstream failures or a stuck consumer. Diagnose lag together with throughput, errors, rebalance history and sink health; do not scale blindly.

Mental model

Event → topic → partition → offset → consumer-group position → side effect. Reliability failures often happen at the boundaries between those states.

Common traps

- Assuming a topic has one global order.
- Treating consumed as deleted.
- Committing progress before durable work without understanding loss risk.
- Saying exactly once without naming the system boundary.

DE14-12 - Practice and proof

PART 2 - GUIDED → SOLO → VALIDATE → EXPLAIN

Guided reproduction

Use lag_by_partition to identify which partition should be investigated first.

Independent task

Diagnose rising lag with a five-step evidence-first incident sequence.

Validate

Save event/key/topic/partition/offset or group/checkpoint evidence appropriate to the lesson, plus one independent control. If a retry/restart is involved, prove whether output duplicates or loss can occur.

Recover

Restate the event and progress boundary. Inspect the first incorrect partition/offset/group/sink state. Preserve the failed attempt. Open protected Review only after the attempt exists.

Explain back

Explain lag, failures and streaming operations in 60-90 seconds, including one failure mode and one validation.

Fresh changed retry

Changed incident: lag rises only on one partition; diagnose hot-key/downstream possibilities.

Mastered only when

Changed task passes without reopening Review, evidence is saved, and you can defend the reliability boundary. No percentage score infers mastery.